Operating Systems
Week 8 Lecture 8

Junaid Abdul Wahid

Previous

- Priority based Scheduling
- Windows Scheduling
- System Model
- Deadlock Characterization
- Prevention

Today Discussion

- Synchronization
- Race Condition problem
- Monitors

Background

Processes can execute concurrently

- May be interrupted at any time, partially completing execution

Concurrent access to shared data may result in data inconsistency

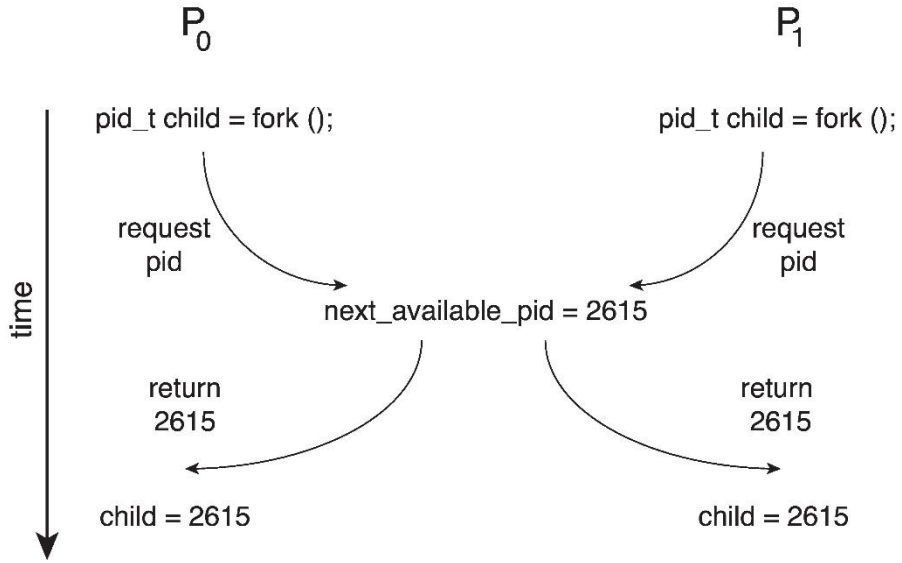
Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes

We illustrated in chapter 4 the problem when we considered the Bounded Buffer problem with use of a counter that is updated concurrently by the producer and consumer,. Which lead to race condition.

Race Condition

Processes P_0 and P_1 are creating child processes using the `fork()` system call

Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)



Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable `next_available_pid` the same pid could be assigned to two different processes!

Critical Section Problem

Consider system of ***n*** processes $\{p_0, p_1, \ldots p_{n-1}\}$

Each process has **critical section** segment of code

- Process may be changing common variables, updating table, writing file, etc.
- When one process in critical section, no other may be in its critical section

Critical section problem is to design protocol to solve this

Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**

Critical Section

General structure of pro

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

Critical-Section Problem (Cont.)

Requirements for solution to critical-section problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the process that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted

   Assume that each process executes at a nonzero speed

   No assumption concerning **relative speed** of the ***n*** processes

Interrupt-based Solution

Entry section: disable interrupts
Exit section: enable interrupts
Will this solve the problem?

- What if the critical section is code that runs for an hour?
- Can some processes starve – never enter their critical section.
- What if there are two CPUs?

Monitors

A high-level abstraction that provides a convenient and effective mechanism for process synchronization
Abstract data type, internal variables only accessible by code within the procedure
Only one process may be active within the monitor at a time
Pseudocode syntax of a monitor:

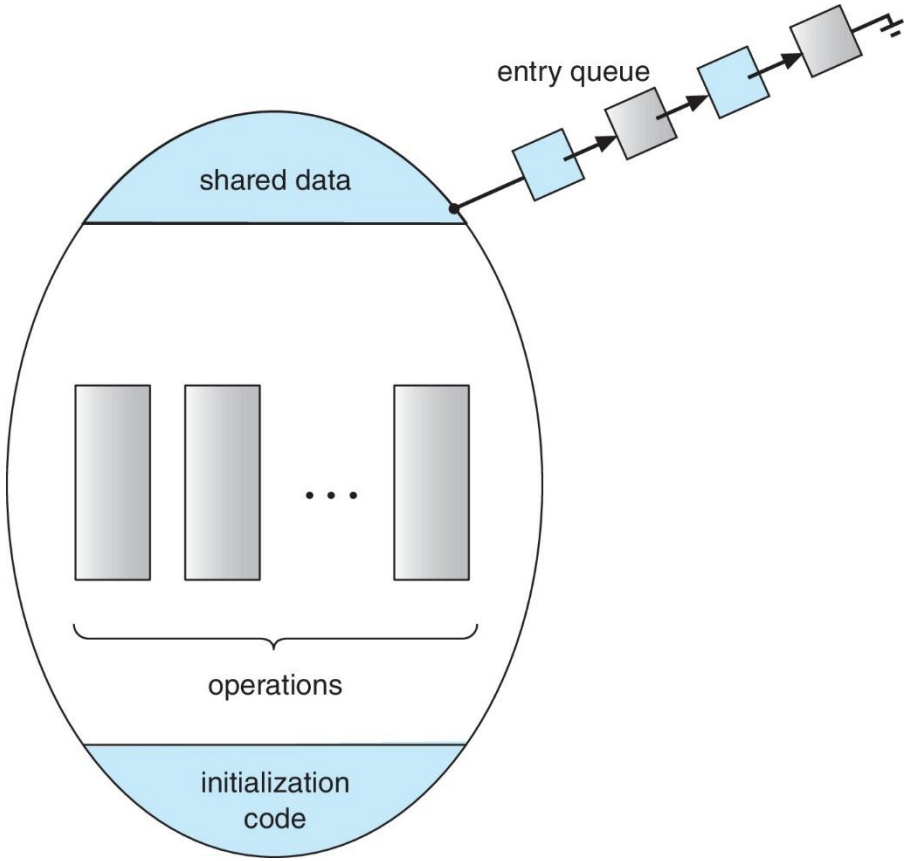
```
monitor monitor-name
{
        // shared variable declarations
        procedure P1 (…) { …. }

        procedure P2 (…) { …. }

        procedure Pn (…) {……}

  initialization code (…) { … }
}
```

Schematic view of a Monitor



Liveness

Processes may have to wait indefinitely while trying to acquire a synchronization tool such as a mutex lock or semaphore.
Waiting indefinitely violates the progress and bounded-waiting criteria discussed at the beginning of this chapter.
Liveness refers to a set of properties that a system must satisfy to ensure processes make progress.
Indefinite waiting is an example of a liveness failure.

Liveness

Deadlock – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
Let *S* and *Q* be two semaphores initialized to 1

P_0	P_1
`wait(S);`	`wait(Q);`
`wait(Q);`	`wait(S);`
...	...
`signal(S);`	`signal(Q);`
`signal(Q);`	`signal(S);`

Consider if P_0 executes wait(S) and P_1 wait(Q). When P_0 executes wait(Q), it must wait until P_1 executes signal(Q)
However, P_1 is waiting until P_0 execute signal(S).
Since these signal() operations will never be executed, P_0 and P_1 are **deadlocked**.

Liveness

Other forms of deadlock:

Starvation – indefinite blocking

- A process may never be removed from the semaphore queue in which it is suspended

Priority Inversion – Scheduling problem when lower-priority process holds a lock needed by higher-priority process

- Solved via **priority-inheritance protocol**

System Model

System consists of resources

Resource types R_1, R_2, . . ., R_m

CPU cycles, memory space, I/O devices

Each resource type R_i has W_i instances.

Each process utilizes a resource as follows:

request

use

release

Deadlock with Semaphores

Data:

- A semaphore s_1 initialized to 1
- A semaphore s_2 initialized to 1

Two threads T_1 and T_2

T_1:

```
wait(s1)
wait(s2)
```

T_2:

```
wait(s2)
wait(s1)
```

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

Mutual exclusion: only one thread at a time can use a resource

Hold and wait: a thread holding at least one resource is waiting to acquire additional resources held by other threads

No preemption: a resource can be released only voluntarily by the thread holding it, after that thread has completed its task

Circular wait: there exists a set $\{T_0, T_1, \ldots, T_n\}$ of waiting threads such that T_0 is waiting for a resource that is held by T_1, T_1 is waiting for a resource that is held by T_2, …, T_{n-1} is waiting for a resource that is held by T_n, and T_n is waiting for a resource that is held by T_0.

Resource-Allocation Graph

A set of vertices V and a set of edges E.

V is partitioned into two types:

$T = \{T_1, T_2, \ldots, T_n\}$, the set consisting of all the threads in the system.

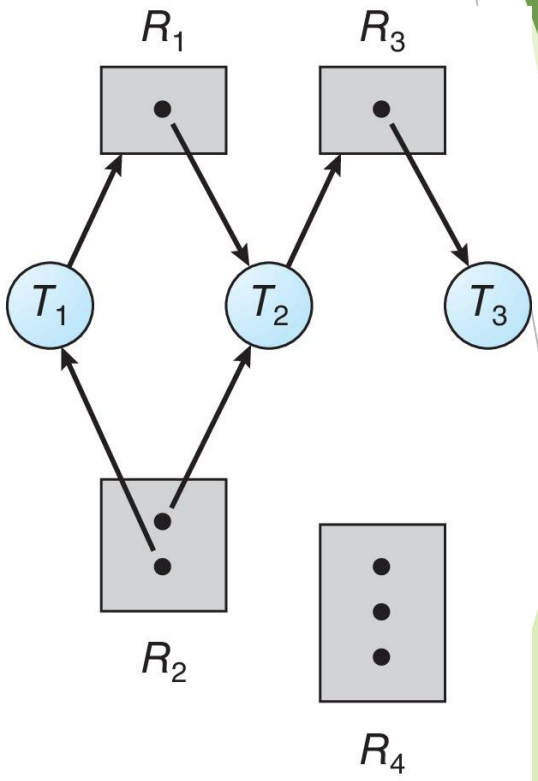
$R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system

request edge – directed edge $T_i \rightarrow R_j$

assignment edge – directed edge $R_j \rightarrow T_i$

Resource Allocation Graph Example

One instance of R_1
Two instances of R_2
One instance of R_3
Three instance of R_4
T_1 holds one instance of R_2 and is waiting for an instance of R_1
T_2 holds one instance of R_1, one instance of R_2, and is waiting for an instance of R_3
T_3 is holds one instance of R_3



Basic Facts

If graph contains no cycles $\Rightarrow$ no deadlock

If graph contains a cycle $\Rightarrow$

- if only one instance per resource type, then deadlock
- if several instances per resource type, possibility of deadlock

Methods for Handling Deadlocks

Ensure that the system will **never** enter a deadlock state:

- Deadlock prevention
- Deadlock avoidance

Allow the system to enter a deadlock state and then recover

Ignore the problem and pretend that deadlocks never occur in the system.

Deadlock Prevention

Invalidate one of the four necessary conditions for deadlock:

Mutual Exclusion – not required for sharable resources (e.g., read-only files); must hold for non-sharable resources

Hold and Wait – must guarantee that whenever a thread requests a resource, it does not hold any other resources

Require threads to request and be allocated all its resources before it begins execution or allow thread to request resources only when the thread has none allocated to it.

Low resource utilization; starvation possible

Deadlock Prevention (Cont.)

No Preemption:

If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released

Preempted resources are added to the list of resources for which the thread is waiting

Thread will be restarted only when it can regain its old resources, as well as the new ones that it is requesting

Circular Wait:

Impose a total ordering of all resource types, and require that each thread requests resources in an increasing order of enumeration